



RÉFÉRENCE COMPLÈTE

Les fonctions PHP

Toutes les fonctions essentielles, par catégorie,
avec explications et exemples d'utilisation

Guide complet — Sonatel Academy
Préparé pour Khadija — Dakar, Sénégal

1. Fonctions sur les chaînes de caractères (Strings)

Les chaînes de caractères (strings) sont des textes. PHP offre une très grande collection de fonctions pour les manipuler.

Longueur et comptage

Fonction	Description et exemple
<code>strlen(\$str)</code>	Retourne le nombre de caractères. <code>strlen("Dakar")</code> → 5
<code>mb_strlen(\$str)</code>	Compte les caractères multi-octets (lettres accentuées, arabes...). À préférer pour le français.
<code>str_word_count(\$str)</code>	Compte le nombre de mots. <code>str_word_count("Bonjour Khadija")</code> → 2
<code>substr_count(\$str,\$sub)</code>	Compte combien de fois une sous-chaîne apparaît. <code>substr_count("banana","a")</code> → 3

Changer la casse (majuscules / minuscules)

Fonction	Description et exemple
<code>strtoupper(\$str)</code>	Met tout en MAJUSCULES. <code>strtoupper("bonjour")</code> → "BONJOUR"
<code>strtolower(\$str)</code>	Met tout en minuscules. <code>strtolower("BONJOUR")</code> → "bonjour"
<code>ucfirst(\$str)</code>	Met la première lettre en majuscule. <code>ucfirst("khadija")</code> → "Khadija"
<code>ucwords(\$str)</code>	Met la première lettre de chaque mot en majuscule. <code>ucwords("bonjour monde")</code> → "Bonjour Monde"
<code>mb_strtoupper(\$str)</code>	Même chose que <code>strtoupper</code> mais gère les caractères accentués (é→É).

Rechercher dans une chaîne

Fonction	Description et exemple
<code>strpos(\$str,\$search)</code>	Retourne la position (index) de la première occurrence. <code>strpos("Dakar","a")</code> → 1. Retourne false si absent.
<code>strrpos(\$str,\$search)</code>	Position de la DERNIÈRE occurrence. <code>strrpos("banana","a")</code> → 5
<code>stripos(\$str,\$search)</code>	Comme <code>strpos</code> mais insensible à la casse (cherche sans distinction maj/min).
<code>str_contains(\$str,\$sub)</code>	Retourne true si la chaîne contient la sous-chaîne. <code>str_contains("Dakar","aka")</code> → true
<code>str_starts_with(\$str,\$p)</code>	Retourne true si la chaîne commence par le préfixe donné.
<code>str_ends_with(\$str,\$s)</code>	Retourne true si la chaîne se termine par le suffixe donné.

```
<?php
    $texte = "Bienvenue à Sonatel Academy";

    echo strlen($texte);           // 27
    echo strpos($texte, "Sonatel"); // 14
    echo str_contains($texte, "Academy") ? "oui" : "non"; // oui
    echo strtoupper($texte);       // BIENVENUE À SONATEL ACADEMY
?>
```

1. Fonctions strings (suite)

Extraire une partie d'une chaîne

Fonction	Description et exemple
<code>substr(\$str,\$start,\$len)</code>	Extrait une portion. <code>substr("Bonjour",3,4)</code> → "jour". Start peut être négatif (part de la fin).
<code>mb_substr(\$str,\$s,\$l)</code>	Même chose mais gère les caractères accentués correctement.
<code>strstr(\$str,\$search)</code>	Retourne la partie de la chaîne à partir de la première occurrence de la recherche.

Remplacer dans une chaîne

Fonction	Description et exemple
<code>str_replace(\$s,\$r,\$str)</code>	Remplace toutes les occurrences de \$s par \$r dans \$str. <code>str_replace("a","o","banana")</code> → "bonono"
<code>str_ireplace(\$s,\$r,\$str)</code>	Même chose mais insensible à la casse.
<code>substr_replace(\$s,\$r,\$p)</code>	Remplace une portion définie par sa position et sa longueur.
<code>preg_replace(\$pat,\$r,\$str)</code>	Remplace selon un motif regex (expression régulière). Très puissant.

Couper, assembler, nettoyer

Fonction	Description et exemple
<code>trim(\$str)</code>	Supprime les espaces (et \n, \t) au début et à la fin.
<code>ltrim(\$str)</code>	Supprime les espaces uniquement à gauche (Left).
<code>rtrim(\$str)</code>	Supprime les espaces uniquement à droite (Right).
<code>str_pad(\$str,\$len,\$pad)</code>	Complète une chaîne jusqu'à une longueur donnée. <code>str_pad("5",3,"0",STR_PAD_LEFT)</code> → "005"
<code>str_repeat(\$str,\$n)</code>	Répète une chaîne n fois. <code>str_repeat("ha",3)</code> → "hahaha"
<code>wordwrap(\$str,\$w,\$br)</code>	Coupe une chaîne longue en lignes de largeur \$w caractères.
<code>chunk_split(\$str,\$n,\$e)</code>	Insère \$e tous les \$n caractères. Utile pour formater du base64.

Diviser et assembler

Fonction	Description et exemple
<code>explode(\$sep,\$str)</code>	Découpe une chaîne en tableau selon un séparateur. <code>explode(",","a,b,c")</code> → ["a","b","c"]
<code>implode(\$sep,\$arr)</code>	Assemble un tableau en chaîne. <code>implode("-",["a","b","c"])</code> → "a-b-c". Alias: join().
<code>str_split(\$str,\$n)</code>	Découpe une chaîne en tableau de morceaux de taille \$n. <code>str_split("ABCD",2)</code> → ["AB","CD"]

```
<?php
    $csv = "Khadija,19,Dakar";
    $tab = explode(",", $csv);
    // ["Khadija", "19", "Dakar"]

    $str = implode(" | ", $tab);
    // "Khadija | 19 | Dakar"

    echo str_replace("Dakar", "Thiès", $str);
    // "Khadija | 19 | Thiès"

?>
```

1. Fonctions strings (fin)

Comparer des chaînes

Fonction	Description et exemple
<code>strcmp(\$s1,\$s2)</code>	Compare deux chaînes. Retourne 0 si égales, négatif si <code>s1 < s2</code> , positif si <code>s1 > s2</code> .
<code>strcasemp(\$s1,\$s2)</code>	Même chose mais insensible à la casse.
<code>similar_text(\$s1,\$s2)</code>	Calcule la similarité entre deux chaînes (nombre de caractères communs).
<code>levenshtein(\$s1,\$s2)</code>	Calcule la "distance" entre deux mots (nombre de modifications pour passer de l'un à l'autre).
<code>soundex(\$str)</code>	Retourne le code phonétique d'un mot (pour trouver des mots qui sonnent pareil).

Formatage et encodage

Fonction	Description et exemple
<code>sprintf(\$format,...)</code>	Formate une chaîne selon un modèle. <code>sprintf("%.2f", 3.14159) → "3.14"</code> . Très puissant.
<code>printf(\$format,...)</code>	Comme <code>sprintf</code> mais affiche directement avec echo.
<code>number_format(\$n,\$d)</code>	Formate un nombre avec séparateurs. <code>number_format(1234567.89,2,".", " ") → "1 234 567,89"</code>
<code>htmlspecialchars(\$str)</code>	Convertit <code>< > " ' &</code> en entités HTML. Essentiel pour la sécurité.
<code>htmlentities(\$str)</code>	Convertit TOUS les caractères spéciaux en entités HTML.
<code>html_entity_decode(\$str)</code>	L'inverse de <code>htmlentities</code> : reconvertit les entités en caractères normaux.
<code>strip_tags(\$str)</code>	Supprime toutes les balises HTML d'une chaîne.
<code>nl2br(\$str)</code>	Remplace les sauts de ligne <code>\n</code> par des <code>
</code> HTML.
<code>base64_encode(\$str)</code>	Encode une chaîne en base64 (pour transmettre des données binaires).
<code>base64_decode(\$str)</code>	Décode une chaîne base64.
<code>urlencode(\$str)</code>	Encode une chaîne pour l'utiliser dans une URL (remplace espaces par + etc.).
<code>urldecode(\$str)</code>	Décode une chaîne encodée pour URL.
<code>md5(\$str)</code>	Calcule le hash MD5 d'une chaîne (non recommandé pour les mots de passe).
<code>sha1(\$str)</code>	Calcule le hash SHA1 (plus sécurisé que MD5 mais toujours pas idéal pour les mdp).
<code>password_hash(\$str,\$algo)</code>	Hache un mot de passe de façon sécurisée. Utiliser <code>PASSWORD_DEFAULT</code> comme algo.
<code>password_verify(\$p,\$h)</code>	Vérifie qu'un mot de passe correspond à son hash. Toujours utiliser avec <code>password_hash</code> .

```
<?php
// Formatage de nombres
echo number_format(1234567.891, 2, ".", " "); // 1 234 567,89
echo sprintf("Il reste %d places sur %d", 3, 10); // Il reste 3 places sur 10

// Sécurité des mots de passe
$hash = password_hash("monMotDePasse", PASSWORD_DEFAULT);
echo password_verify("monMotDePasse", $hash); // true

?>
```

Fonctions mathématiques et numériques

Opérations de base

Fonction	Description et exemple
<code>abs(\$n)</code>	Valeur absolue. <code>abs(-5)</code> → 5
<code>ceil(\$n)</code>	Arrondi au supérieur (toujours vers le haut). <code>ceil(4.1)</code> → 5
<code>floor(\$n)</code>	Arrondi à l'inférieur (toujours vers le bas). <code>floor(4.9)</code> → 4
<code>round(\$n,\$prec)</code>	Arrondi classique. <code>round(4.5)</code> → 5. <code>round(3.14159,2)</code> → 3.14
<code>fmod(\$a,\$b)</code>	Reste de la division pour les décimaux. <code>fmod(10.5,3.5)</code> → 0
<code>intdiv(\$a,\$b)</code>	Division entière. <code>intdiv(10,3)</code> → 3 (sans le reste)
<code>pow(\$base,\$exp)</code>	Puissance. <code>pow(2,8)</code> → 256
<code>sqrt(\$n)</code>	Racine carrée. <code>sqrt(16)</code> → 4
<code>max(\$a,\$b,...)</code>	Retourne la valeur la plus grande. <code>max(3,8,1,5)</code> → 8. Fonctionne aussi avec un tableau.
<code>min(\$a,\$b,...)</code>	Retourne la valeur la plus petite. <code>min(3,8,1,5)</code> → 1
<code>array_sum(\$arr)</code>	Calcule la somme de tous les éléments d'un tableau numérique.
<code>array_product(\$arr)</code>	Calcule le produit de tous les éléments d'un tableau numérique.

Aléatoire et constantes

Fonction	Description et exemple
<code>rand(\$min,\$max)</code>	Nombre entier aléatoire entre min et max. <code>rand(1,6)</code> → simule un dé.
<code>mt_rand(\$min,\$max)</code>	Comme rand() mais plus rapide et plus aléatoire (Mersenne Twister).
<code>random_int(\$min,\$max)</code>	Nombre aléatoire cryptographiquement sécurisé. À préférer pour la sécurité.
<code>M_PI</code>	Constante π (pi) ≈ 3.14159265...
<code>M_E</code>	Constante e (nombre d'Euler) ≈ 2.71828...
<code>PHP_INT_MAX</code>	Plus grand entier possible en PHP.

Conversion et vérification de nombres

Fonction	Description et exemple
<code>intval(\$var)</code>	Convertit en entier. <code>intval("42abc")</code> → 42
<code>floatval(\$var)</code>	Convertit en nombre décimal. Alias: doubleval().
<code>is_int(\$var)</code>	Vérifie si c'est un entier. Retourne true ou false.
<code>is_float(\$var)</code>	Vérifie si c'est un décimal.
<code>is_numeric(\$var)</code>	Vérifie si la valeur est numérique (même si c'est une chaîne comme "42").
<code>number_format(\$n,\$d,\$dec,\$mil)</code>	Formate un nombre avec décimales et séparateurs de milliers.
<code>decbin(\$n)</code>	Convertit un décimal en binaire. <code>decbin(10)</code> → "1010"
<code>bindec(\$str)</code>	Convertit du binaire en décimal. <code>bindec("1010")</code> → 10
<code>dechex(\$n)</code>	Convertit en hexadécimal. <code>dechex(255)</code> → "ff"
<code>decoct(\$n)</code>	Convertit en octal.

3. Fonctions sur les tableaux (Arrays)

Créer et inspecter un tableau

Fonction	Description et exemple
<code>count(\$arr)</code>	Nombre d'éléments. <code>count([1,2,3])</code> → 3
<code>sizeof(\$arr)</code>	Alias de <code>count()</code> .
<code>is_array(\$var)</code>	Vérifie si c'est un tableau. Retourne true ou false.
<code>in_array(\$val,\$arr)</code>	Vérifie si une valeur existe dans le tableau. Retourne true ou false.
<code>array_key_exists(\$k,\$arr)</code>	Vérifie si une clé existe dans le tableau.
<code>isset(\$arr[\$k])</code>	Vérifie si une clé existe ET que sa valeur n'est pas null.
<code>array_keys(\$arr)</code>	Retourne toutes les clés du tableau.
<code>array_values(\$arr)</code>	Retourne toutes les valeurs en réindexant de 0.
<code>array_count_values(\$arr)</code>	Compte combien de fois chaque valeur apparaît dans le tableau.

Ajouter et supprimer des éléments

Fonction	Description et exemple
<code>array_push(\$arr,\$val)</code>	Ajoute un ou plusieurs éléments à la FIN. Équivalent à <code>\$arr[] = \$val;</code>
<code>array_pop(\$arr)</code>	Supprime et retourne le DERNIER élément.
<code>array_unshift(\$arr,\$val)</code>	Ajoute un ou plusieurs éléments au DÉBUT (réindexe les clés).
<code>array_shift(\$arr)</code>	Supprime et retourne le PREMIER élément (réindexe les clés).
<code>unset(\$arr[\$k])</code>	Supprime un élément par sa clé (ne réindexe PAS).
<code>array_splice(\$arr,\$s,\$n)</code>	Supprime \$n éléments à partir de la position \$s et peut les remplacer.

Trier un tableau

Fonction	Description et exemple
<code>sort(\$arr)</code>	Trie les valeurs par ordre croissant. Réinitialise les clés numériques.
<code>rsort(\$arr)</code>	Trie les valeurs par ordre décroissant.
<code>asort(\$arr)</code>	Trie les valeurs croissant en CONSERVANT les clés associatives.
<code>arsort(\$arr)</code>	Trie les valeurs décroissant en conservant les clés.
<code>ksort(\$arr)</code>	Trie par clés croissantes.
<code>krsort(\$arr)</code>	Trie par clés décroissantes.
<code>usort(\$arr,\$fn)</code>	Trie avec une fonction de comparaison personnalisée. Très flexible.
<code>uasort(\$arr,\$fn)</code>	Comme <code>usort</code> mais conserve les clés associatives.
<code>shuffle(\$arr)</code>	Mélange les éléments aléatoirement.
<code>array_reverse(\$arr)</code>	Inverse l'ordre des éléments. Retourne un nouveau tableau.

Fonctions sur les tableaux (suite)

Transformer un tableau

Fonction	Description et exemple
<code>array_map(\$fn,\$arr)</code>	Applique une fonction à chaque élément et retourne le nouveau tableau.
<code>array_filter(\$arr,\$fn)</code>	Garde seulement les éléments pour lesquels la fonction retourne true.
<code>array_reduce(\$arr,\$fn,\$init)</code>	Réduit le tableau à une seule valeur (somme, produit, concaténation...).
<code>array_walk(\$arr,\$fn)</code>	Applique une fonction à chaque élément en le modifiant en place (modifie l'original).
<code>array_flip(\$arr)</code>	Échange les clés et les valeurs.
<code>array_unique(\$arr)</code>	Supprime les valeurs en double, conserve les clés.
<code>array_combine(\$keys,\$vals)</code>	Crée un tableau associatif en combinant un tableau de clés et un tableau de valeurs.

Extraire et fusionner

Fonction	Description et exemple
<code>array_slice(\$arr,\$s,\$n)</code>	Extrait une portion du tableau (index de départ, nombre d'éléments).
<code>array_merge(\$a,\$b)</code>	Fusionne deux tableaux. Les clés numériques sont réindexées.
<code>array_merge_recursive(\$a,\$b)</code>	Fusion récursive : les valeurs avec mêmes clés sont regroupées dans un tableau.
<code>array_chunk(\$arr,\$n)</code>	Divise un tableau en morceaux de taille \$n. Retourne un tableau de tableaux.
<code>array_column(\$arr,\$col)</code>	Extrait une colonne d'un tableau multidimensionnel. Très utile avec les résultats MySQL.
<code>compact(\$v1,\$v2,...)</code>	Crée un tableau associatif à partir de noms de variables. L'inverse de extract().
<code>extract(\$arr)</code>	Transforme un tableau associatif en variables locales. L'inverse de compact().

Rechercher dans un tableau

Fonction	Description et exemple
<code>array_search(\$val,\$arr)</code>	Retourne la clé de la première valeur trouvée, ou false.
<code>array_diff(\$a,\$b)</code>	Retourne les éléments présents dans \$a mais PAS dans \$b.
<code>array_intersect(\$a,\$b)</code>	Retourne les éléments communs entre \$a et \$b.
<code>array_diff_key(\$a,\$b)</code>	Comme array_diff mais compare les CLÉS au lieu des valeurs.

```
<?php
    $etudiants = [
        ["nom"=>"Khadija", "note"=>14],
        ["nom"=>"Moussa", "note"=>8],
        ["nom"=>"Fatou", "note"=>17],
    ];

    // Extraire uniquement les noms
    $noms = array_column($etudiants, "nom");
    // ["Khadija", "Moussa", "Fatou"]

    // Garder seulement ceux qui ont la moyenne
    $recus = array_filter($etudiants, fn($e) => $e["note"] >= 10);
    // Khadija et Fatou
?>
```

4. Fonctions de date et heure

Obtenir la date et l'heure actuelles

Fonction	Description et exemple
<code>time()</code>	Retourne le timestamp Unix actuel (nombre de secondes depuis le 1er jan 1970).
<code>microtime(true)</code>	Retourne le timestamp avec les microsecondes. Utile pour mesurer les performances.
<code>date(\$format,\$ts)</code>	Formate une date. Si \$ts omis, utilise l'heure actuelle.
<code>date_create(\$str)</code>	Crée un objet DateTime à partir d'une chaîne. Alias: <code>new DateTime()</code> .
<code>date_format(\$obj,\$fmt)</code>	Formate un objet DateTime.

Les formats de date() les plus utilisés

Format	Signification	Exemple
<code>d</code>	Jour du mois (01 à 31)	<code>date("d")</code> → "26"
<code>m</code>	Mois en chiffres (01 à 12)	<code>date("m")</code> → "06"
<code>Y</code>	Année sur 4 chiffres	<code>date("Y")</code> → "2026"
<code>y</code>	Année sur 2 chiffres	<code>date("y")</code> → "26"
<code>H</code>	Heure 24h (00 à 23)	<code>date("H")</code> → "14"
<code>i</code>	Minutes (00 à 59)	<code>date("i")</code> → "35"
<code>s</code>	Secondes (00 à 59)	<code>date("s")</code> → "07"
<code>D</code>	Jour en anglais abrégé	<code>date("D")</code> → "Fri"
<code>l</code>	Jour en anglais complet	<code>date("l")</code> → "Friday"
<code>N</code>	Numéro du jour (1=lundi, 7=dim)	<code>date("N")</code> → "5"
<code>w</code>	Numéro de semaine dans l'année	<code>date("W")</code> → "26"

Manipuler les dates

Fonction	Description et exemple
<code>mktime(\$h,\$m,\$s,\$mo,\$d,\$y)</code>	Crée un timestamp à partir de composantes date/heure.
<code>strtotime(\$str)</code>	Convertit une description textuelle en timestamp. <code>strtotime("+1 week")</code> → dans 7 jours.
<code>date_diff(\$d1,\$d2)</code>	Calcule la différence entre deux objets DateTime. Retourne un objet DateInterval.
<code>date_add(\$date,\$interval)</code>	Ajoute un intervalle à une date.
<code>checkdate(\$m,\$d,\$y)</code>	Vérifie si une date est valide. <code>checkdate(2,30,2024)</code> → false (il n'y a pas de 30 février).

```
<?php
    echo date("d/m/Y H:i:s");           // 26/06/2026 14:35:07
    echo date("d/m/Y", strtotime("+30 days")); // dans 30 jours

    $debut = date_create("2025-09-01");
    $fin    = date_create("2026-06-30");
    $diff   = date_diff($debut, $fin);
    echo $diff->m . " mois";           // Durée en mois

?>
```


5. Fonctions sur les fichiers et dossiers

Lire et écrire des fichiers

Fonction	Description et exemple
<code>file_get_contents(\$path)</code>	Lit tout le contenu d'un fichier en une seule chaîne. Très simple à utiliser.
<code>file_put_contents(\$path,\$data)</code>	Écrit des données dans un fichier (crée le fichier si absent). Retourne le nb d'octets écrits.
<code>file(\$path)</code>	Lit un fichier et retourne chaque ligne comme un élément de tableau.
<code>fopen(\$path,\$mode)</code>	Ouvre un fichier et retourne un pointeur. Modes : "r" (lecture), "w" (écriture), "a" (ajout).
<code>fread(\$handle,\$length)</code>	Lit \$length octets depuis un fichier ouvert avec fopen().
<code>fwrite(\$handle,\$str)</code>	Écrit une chaîne dans un fichier ouvert avec fopen().
<code>fgets(\$handle)</code>	Lit une ligne depuis un fichier ouvert.
<code>feof(\$handle)</code>	Vérifie si on a atteint la fin du fichier (End Of File).
<code>fclose(\$handle)</code>	Ferme un fichier ouvert avec fopen(). Toujours fermer après utilisation.

Informations sur les fichiers

Fonction	Description et exemple
<code>file_exists(\$path)</code>	Vérifie si un fichier ou dossier existe. Retourne true ou false.
<code>is_file(\$path)</code>	Vérifie si c'est un fichier (pas un dossier).
<code>is_dir(\$path)</code>	Vérifie si c'est un dossier.
<code>filesize(\$path)</code>	Retourne la taille du fichier en octets.
<code>filemtime(\$path)</code>	Retourne la date de dernière modification (timestamp).
<code>pathinfo(\$path)</code>	Retourne un tableau avec dirname, basename, extension, filename.
<code>basename(\$path)</code>	Retourne le nom du fichier depuis un chemin. <code>basename("/var/www/index.php")</code> → "index.php"
<code>dirname(\$path)</code>	Retourne le dossier parent. <code>dirname("/var/www/index.php")</code> → "/var/www"
<code>realpath(\$path)</code>	Retourne le chemin absolu réel d'un fichier (résout les ..).

Manipuler fichiers et dossiers

Fonction	Description et exemple
<code>copy(\$src,\$dst)</code>	Copie un fichier.
<code>rename(\$old,\$new)</code>	Renomme ou déplace un fichier.
<code>unlink(\$path)</code>	Supprime un fichier. Attention : irréversible !
<code>mkdir(\$path,\$perm)</code>	Crée un dossier. <code>mkdir("uploads",0755)</code>
<code>rmdir(\$path)</code>	Supprime un dossier vide.
<code>scandir(\$path)</code>	Retourne la liste des fichiers et dossiers dans un répertoire.
<code>glob(\$pattern)</code>	Retourne les fichiers correspondant à un motif. <code>glob("*.php")</code> → tous les fichiers .php

6. Fonctions MySQL (MySQLi) et de filtrage

Connexion et requêtes MySQLi

Fonction	Description
<code>mysqli_connect(\$h,\$u,\$p,\$db)</code>	Crée une connexion à la base MySQL. Retourne un objet connexion ou false.
<code>mysqli_connect_error()</code>	Retourne le message d'erreur de la dernière tentative de connexion.
<code>mysqli_query(\$conn,\$sql)</code>	Exécute une requête SQL. Retourne un résultat ou true/false.
<code>mysqli_fetch_assoc(\$res)</code>	Retourne la ligne suivante du résultat sous forme de tableau associatif.
<code>mysqli_fetch_row(\$res)</code>	Retourne la ligne suivante sous forme de tableau indexé (indices numériques).
<code>mysqli_fetch_all(\$res,\$mode)</code>	Retourne toutes les lignes d'un coup. Mode: MYSQLI_ASSOC ou MYSQLI_NUM.
<code>mysqli_num_rows(\$res)</code>	Retourne le nombre de lignes d'un résultat SELECT.
<code>mysqli_affected_rows(\$conn)</code>	Nombre de lignes affectées par INSERT, UPDATE ou DELETE.
<code>mysqli_insert_id(\$conn)</code>	Retourne l'ID auto-incrémenté du dernier INSERT.
<code>mysqli_real_escape_string(\$c,\$s)</code>	Échappe les caractères dangereux pour prévenir les injections SQL.
<code>mysqli_prepare(\$conn,\$sql)</code>	Prépare une requête avec des paramètres (méthode la plus sécurisée).
<code>mysqli_close(\$conn)</code>	Ferme la connexion à la base de données.

Fonctions de filtrage et validation

Fonction	Description
<code>filter_var(\$val,\$filter)</code>	Valide ou nettoie une variable selon un filtre prédéfini. Très puissant.
<code>filter_input(\$type,\$name,\$f)</code>	Récupère et filtre une variable externe (GET, POST, COOKIE...).

Les filtres les plus utilisés avec filter_var()

Filtre	Usage
<code>FILTER_VALIDATE_EMAIL</code>	Vérifie si c'est une adresse email valide.
<code>FILTER_VALIDATE_URL</code>	Vérifie si c'est une URL valide.
<code>FILTER_VALIDATE_INT</code>	Vérifie si c'est un entier valide.
<code>FILTER_VALIDATE_FLOAT</code>	Vérifie si c'est un nombre décimal valide.
<code>FILTER_VALIDATE_IP</code>	Vérifie si c'est une adresse IP valide.
<code>FILTER_SANITIZE_STRING</code>	Supprime les balises HTML et les caractères spéciaux.
<code>FILTER_SANITIZE_NUMBER_INT</code>	Supprime tout sauf les chiffres et les signes + et -.

```
<?php
    $email = "khadija@sonatel.sn";
    if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "Email valide !";
    }

    $age = filter_var("25abc", FILTER_SANITIZE_NUMBER_INT);
    echo $age;    // 25 (les lettres sont supprimées)
?>
```

7. Fonctions sur les variables et les types

Vérifier le type d'une variable

Fonction	Description et exemple
<code>gettype(\$var)</code>	Retourne le type sous forme de chaîne : "string", "integer", "double", "boolean", "array", "object", "NULL".
<code>is_string(\$var)</code>	Vérifie si c'est une chaîne de caractères.
<code>is_int(\$var)</code>	Vérifie si c'est un entier. Alias: <code>is_integer()</code> , <code>is_long()</code> .
<code>is_float(\$var)</code>	Vérifie si c'est un décimal. Alias: <code>is_double()</code> .
<code>is_bool(\$var)</code>	Vérifie si c'est un booléen (true ou false).
<code>is_array(\$var)</code>	Vérifie si c'est un tableau.
<code>is_object(\$var)</code>	Vérifie si c'est un objet (instance de classe).
<code>is_null(\$var)</code>	Vérifie si la valeur est NULL.
<code>is_numeric(\$var)</code>	Vérifie si la valeur est numérique (entier, décimal, ou chaîne numérique).
<code>is_callable(\$var)</code>	Vérifie si la variable peut être appelée comme une fonction.

Convertir le type d'une variable

Fonction	Description et exemple
<code>intval(\$var)</code>	Convertit en entier. <code>intval("42abc")</code> → 42
<code>floatval(\$var)</code>	Convertit en décimal. <code>floatval("3.14abc")</code> → 3.14
<code>strval(\$var)</code>	Convertit en chaîne de caractères. <code>strval(42)</code> → "42"
<code>boolval(\$var)</code>	Convertit en booléen. 0, "", null, [] donnent false. Tout le reste donne true.
<code>settype(\$var,\$type)</code>	Modifie directement le type d'une variable. <code>settype(\$x,"integer")</code>

Vérifier l'existence d'une variable

Fonction	Description et exemple
<code>isset(\$var)</code>	Vérifie si une variable existe ET n'est pas NULL. Retourne true ou false.
<code>empty(\$var)</code>	Retourne true si la variable est vide : 0, "", "0", null, [], false.
<code>unset(\$var)</code>	Détruit une variable, elle cesse d'exister.
<code>var_dump(\$var)</code>	Affiche le type ET la valeur d'une variable. Essentiel pour déboguer.
<code>var_export(\$var)</code>	Affiche la représentation PHP d'une variable (peut être réutilisée dans du code).
<code>print_r(\$var)</code>	Affiche une représentation lisible d'une variable ou d'un tableau. Idéal pour déboguer.

```
<?php
$data = ["prenom" => "Khadija", "age" => 19];
var_dump($data);
// array(2) { ["prenom"]=> string(7) "Khadija" ["age"]=> int(19) }

print_r($data);
// Array ( [prenom] => Khadija [age] => 19 )

echo gettype($data["age"]);    // integer

?>
```

8. Expressions régulières (Regex) et fonctions avancées

QU'EST-CE QU'UNE EXPRESSION RÉGULIÈRE ?

Une expression régulière (regex) est un motif (pattern) qui décrit un format de texte. On l'utilise pour chercher, valider ou remplacer du texte selon des règles complexes (ex : "un email valide", "un numéro de téléphone à 9 chiffres", etc.).

Fonctions regex (preg_*)

Fonction	Description
<code>preg_match(\$pat,\$str)</code>	Vérifie si le motif \$pat est trouvé dans \$str. Retourne 1 (trouvé) ou 0.
<code>preg_match_all(\$pat,\$str,\$m)</code>	Trouve TOUTES les occurrences et les met dans \$m. Retourne le nombre de correspondances.
<code>preg_replace(\$pat,\$r,\$str)</code>	Remplace les correspondances du motif par \$r dans \$str.
<code>preg_split(\$pat,\$str)</code>	Découpe une chaîne selon un motif regex (plus puissant qu'explode()).
<code>preg_grep(\$pat,\$arr)</code>	Filtre un tableau et retourne les éléments qui correspondent au motif.

Les motifs regex de base

Motif	Signification	Exemple
<code>\d</code>	Un chiffre (0-9)	<code>/\d+/</code> → un ou plusieurs chiffres
<code>\w</code>	Un caractère alphanumérique	<code>/\w+/</code> → un ou plusieurs caractères
<code>\s</code>	Un espace (espace, tab, newline)	<code>/\s+/</code> → un ou plusieurs espaces
<code>.</code>	N'importe quel caractère	<code>/./</code> → un seul caractère quelconque
<code>^</code>	Début de chaîne	<code>/^Bonjour/</code> → commence par "Bonjour"
<code>\$</code>	Fin de chaîne	<code>/monde\$/</code> → se termine par "monde"
<code>*</code>	0 ou plusieurs fois	<code>/ab*/</code> → a, ab, abb, abbb...
<code>+</code>	1 ou plusieurs fois	<code>/ab+/</code> → ab, abb, abbb...
<code>{n,m}</code>	Entre n et m fois	<code>/\d{8,9}/</code> → 8 à 9 chiffres

```
<?php
// Valider un numéro de téléphone sénégalais (9 chiffres)
$tel = "776543210";
if (preg_match('/^(77|78|76|70)\d{7}$/', $tel)) {
    echo "Numéro valide";
}

// Remplacer tous les espaces multiples par un seul espace
$texte = preg_replace('/\s+/', ' ', "Bonjour  tout  le  monde");
// "Bonjour tout le monde"

?>
```

Fonctions système, JSON et divers essentiels

Fonctions JSON

Fonction	Description et exemple
<code>json_encode(\$var)</code>	Convertit une variable PHP (tableau, objet) en chaîne JSON. Très utilisé pour les APIs.
<code>json_decode(\$str,\$assoc)</code>	Convertit une chaîne JSON en variable PHP. Si <code>\$assoc=true</code> , retourne un tableau associatif.
<code>json_last_error()</code>	Retourne le code d'erreur du dernier <code>json_encode/decode</code> . <code>JSON_ERROR_NONE</code> = pas d'erreur.

```
<?php
    $data = ["prenom" => "Khadija", "age" => 19];

    $json = json_encode($data);
    // '{"prenom":"Khadija","age":19}'

    $retour = json_decode($json, true);
    // ["prenom" => "Khadija", "age" => 19]

?>
```

Fonctions de contrôle du script

Fonction	Description
<code>die(\$msg)</code>	Arrête le script et affiche <code>\$msg</code> . Alias: <code>exit()</code> .
<code>exit(\$code)</code>	Arrête le script. Si <code>\$code</code> est un entier, c'est le code de sortie (0 = succès).
<code>header(\$str)</code>	Envoie un en-tête HTTP brut. Utilisé pour les redirections et le type de contenu.
<code>headers_sent()</code>	Vérifie si les en-têtes HTTP ont déjà été envoyés.
<code>ob_start()</code>	Démarre la mise en tampon de la sortie (Output Buffering).
<code>ob_get_clean()</code>	Retourne le contenu du tampon et le vide.
<code>sleep(\$n)</code>	Met le script en pause <code>\$n</code> secondes.
<code>usleep(\$n)</code>	Met le script en pause <code>\$n</code> microsecondes.

Fonctions d'information système

Fonction	Description
<code>phpversion()</code>	Retourne la version actuelle de PHP.
<code>php_uname()</code>	Retourne les informations sur le système d'exploitation.
<code>memory_get_usage()</code>	Retourne la mémoire utilisée par le script en octets.
<code>memory_get_peak_usage()</code>	Retourne le pic de mémoire utilisé.
<code>microtime(true)</code>	Timestamp en secondes avec microsecondes. Utile pour mesurer la durée d'exécution.
<code>defined(\$const)</code>	Vérifie si une constante est définie.
<code>define(\$name,\$val)</code>	Définit une constante immuable. <code>define("TVA", 0.18);</code>
<code>constant(\$name)</code>	Retourne la valeur d'une constante à partir de son nom sous forme de chaîne.
<code>error_reporting(\$level)</code>	Définit le niveau de signalement des erreurs.
<code>set_error_handler(\$fn)</code>	Définit une fonction personnalisée pour gérer les erreurs.

Fonctions mathématiques supplémentaires

Fonction	Description
<code>log(\$n,\$base)</code>	Logarithme. <code>log(M_E) → 1</code> . <code>log(100,10) → 2</code>
<code>exp(\$n)</code>	Exponentielle e^n. <code>exp(1) → 2.718...</code>
<code>sin(\$n) cos(\$n) tan(\$n)</code>	Fonctions trigonométriques (en radians).

`pi()` Retourne la valeur de π . Équivalent à la constante `M_PI`.

`base_convert($n,$from,$to)` Convertit un nombre d'une base à une autre. `base_convert("ff",16,10) → "255"`

Résumé : les catégories de fonctions PHP

Catégorie	Fonctions clés à retenir
Chaînes	strlen, strtoupper, strtolower, strpos, str_contains, substr, str_replace, explode, implode, trim, htmlspecialchars, sprintf, password_hash
Tableaux	count, in_array, array_push, array_pop, sort, array_map, array_filter, array_reduce, array_merge, array_slice, array_unique, array_column
Nombres	abs, ceil, floor, round, rand, max, min, pow, sqrt, number_format, is_numeric
Dates	time, date, strtotime, mktime, date_create, date_diff, checkdate
Fichiers	file_get_contents, file_put_contents, file_exists, fopen, fread, fwrite, fclose, copy, rename, unlink, mkdir
MySQL	mysqli_connect, mysqli_query, mysqli_fetch_assoc, mysqli_num_rows, mysqli_prepare, mysqli_close
Variables	isset, empty, unset, gettype, var_dump, print_r, is_string, is_int, intval
Regex	preg_match, preg_replace, preg_split
JSON	json_encode, json_decode
Système	header, die, exit, define, phpversion, memory_get_usage

🔔 La phrase à retenir pour ta présentation

"PHP dispose de plus de 1000 fonctions intégrées organisées par catégories : les fonctions string pour manipuler le texte, les fonctions array pour les tableaux, les fonctions mathématiques, les fonctions de date, de fichiers, de base de données MySQL, et bien d'autres. La documentation officielle php.net est la référence complète et gratuite pour les consulter toutes."

RESSOURCE OFFICIELLE

Toutes les fonctions PHP sont documentées sur <https://www.php.net/manual/fr/> avec des exemples, des commentaires de la communauté et des explications en français. C'est la bible du développeur PHP.